



ENSEEIH7 1SN - Groupe GH06

Projet - Calcul Scientifique

Rapport de :
Yan RABEFANIRAKA
Aloïs MEURISSE

Rendu le 07 Mai 2025

Table des matières

1	Méthodes d'itération de sous-espaces	2
1.1	Préambule	2
1.2	Méthode des puissances itérées (<i>Power Method</i>)	2
1.2.1	Comparaison avec la fonction <i>eig</i>	3
1.2.2	Amélioration de l'algorithme en retirant un produit matrice-vecteur	3
1.2.3	Limites de la puissance itérée en complexité temporelle	4
1.3	Extension de la puissance itérée au calcul des sous-espaces propres (<i>Subspace Iteration v_0</i>)	4
1.3.1	Converge de la matrice V	5
1.3.2	Impact du calcul de la décomposition spectrale de H	5
1.3.3	Implémentation de l'algorithme 2	5
1.4	Amélioration en utilisant la projection de <i>Rayleigh-Ritz</i> (<i>Subspace Iteration v_1</i>)	6
1.4.1	Concordance code-algorithme	7
1.5	Accélération grâce à l'Approche par blocs (<i>Subspace Iteration v_2</i>)	7
1.5.1	Coûts de calcul des matrices A^p et $A^p.V$	7
1.5.2	Implémentation de l'approche par bloc	8
1.5.3	Comportement de la méthode selon la valeur de p	8
1.6	Accélération et allègement avec la Méthode de Déflation (<i>Subspace Iteration v_3</i>)	8
1.6.1	Différences dans la qualité des couples propres (qv)	8
1.6.2	Prédictions sur la précision des couples propres avec la méthode de déflation	9
1.6.3	Implémentation de la méthode de déflation	9
1.7	Expérimentations	10
1.7.1	Types de matrices utilisées et comparaison selon la distribution des couples propres	10
1.7.2	Comparaison complète des méthodes implémentées avec celles fournies	11
2	Application à la compression d'images	13
2.1	Question 1	13
2.2	Question 2	13
2.2.1	Fonction <i>eig</i>	13
2.2.2	Méthode de la puissance itérée	14
2.2.3	Subspace Iteration Method, Version 0	14
2.2.4	Subspace Iteration Method, Version 1	15
2.2.5	Subspace Iteration Method, Version 2	16
2.2.6	Subspace Iteration Method, Version 3	17

Chapitre 1

Méthodes d'itération de sous-espaces

1.1 Préambule

Nous savons qu'il n'est pas toujours nécessaire de trouver l'entièreté des couples propres d'une matrice pour en exploiter les informations mais seulement les couples principaux.

Pour cela, nous avons étudié la **méthode des puissances itérées** permettant de calculer le couple propre dominant d'une matrice. Cela nous permet ensuite, à l'aide d'une *déflation*, de trouver les couples propres suivants.

Cependant, cet algorithme n'est pas spécialement performant ni précis. Ainsi, nous allons voir dans ce projet ses limitations et donc comment l'étendre à l'**itération de sous-espaces** afin de l'améliorer, et ce à travers quatre méthodes différentes. Nous comparerons ces méthodes grâce à différentes matrices et détaillerons leur implémentation et les raisons de leur performance.

1.2 Méthode des puissances itérées (*Power Method*)

L'algorithme de la puissance itérée étudié permet de déterminer le vecteur propre associé à la valeur propre dominante en module. Voici l'algorithme de l'implémentation fournie, avec des critères d'arrêts légèrement différents de l'algorithme initial :

Algorithm 0 Puissance itérée v11

- 1: Input : Matrix $A \in \mathbb{R}^{n \times n}$
 - 2: Output : (λ_1, v_1) eigenpair associated to the largest (in module) eigenvalue.
 - 3: $v \in \mathbb{R}^n$ given
 - 4: $z = A \cdot v$
 - 5: $\beta = v^T \cdot z$
 - 6: **repeat**
 - 7: $y = A \cdot v$
 - 8: $v = y / \|y\|$
 - 9: $z = A \cdot v$
 - 10: $\beta = v^T \cdot z$
 - 11: **until** $|\beta \cdot v - z| / |\beta| < \varepsilon$
 - 12: $\lambda_1 = \beta$ and $v_1 = v$
-

1.2.1 Comparaison avec la fonction *eig*

L'implémentation de la méthode de la puissance itérée fournie permet de trouver quelques couples propres, là où la fonction intégrée *eig* de *Matlab* calcule tous les couples propres. Nous pouvons remarquer que malgré cela, l'efficacité temporelle de notre implémentation de départ est bien au-delà de celle de *Matlab*, qui est relativement constante qu'importe le type et la taille des matrices utilisées. :

Matrices		puissance itérée	eig
100 100	Type 1	6×10^{-2}	1×10^{-2}
	Type 2	1×10^{-2}	2×10^{-2}
	Type 3	1×10^{-2}	1×10^{-2}
	Type 4	6×10^{-2}	1×10^{-2}
500 500	Type 1	2.77	6×10^{-2}
	Type 2	1.7×10^{-1}	5×10^{-2}
	Type 3	6.8×10^{-2}	7×10^{-2}
	Type 4	2.995	6×10^{-2}
1000 1000	Type 1	ne converge pas	3.1×10^{-1}
	Type 2	3.7	3.1×10^{-1}
	Type 3	18.45	3×10^{-1}
	Type 4	ne converge pas	3.1×10^{-1}

TABLE 1.1 – Comparaison des performances temporelles en *secondes* entre la puissance itérée fournie et *eig*.

1.2.2 Amélioration de l'algorithme en retirant un produit matrice-vecteur

On remarque que l'algorithme [0] effectue deux fois le produit $A \cdot v$ aux lignes [6] et [9]. En corrigeant ce doublon et en n'ayant qu'un unique produit $A \cdot v$, on obtient une implémentation environ deux fois plus rapide. Ainsi, on obtient l'algorithme suivant :

Algorithm 1 Puissance itérée $v12$ avec un unique produit matrice-vecteur

```
1: Input : Matrix  $A \in \mathbb{R}^{n \times n}$ 
2: Output :  $(\lambda_1, v_1)$  eigenpair associated to the largest (in module) eigenvalue.
3:  $v \in \mathbb{R}^n$  given
4:  $z = A \cdot v$ 
5:  $\beta = v^T \cdot z$ 
6: repeat
7:    $y = z$  {Le changement ici.}
8:    $v = y / \|y\|$ 
9:    $z = A \cdot v$ 
10:   $\beta = v^T \cdot z$ 
11: until  $|\beta \cdot v - z| / |\beta| < \varepsilon$ 
12:  $\lambda_1 = \beta$  and  $v_1 = v$ 
```

On peut vérifier avec le *test_v11v12* la vitesse des deux implémentations pour différents types de matrices :

Matrices		v11	v12
500×500	Type 1	2.78	1.43
	Type 2	1.1×10^{-1}	7×10^{-2}
	Type 3	7.2×10^{-1}	3.7×10^{-1}
	Type 4	3.9	1.994

TABLE 1.2 – Comparaison du temps de calcul moyen entre les deux implémentations de la puissance itérée.

On voit effectivement que la nouvelle version est environ deux fois plus rapide.

1.2.3 Limites de la puissance itérée en complexité temporelle

On remarque vite à partir de ces tableaux comparatifs que les principales limites de la puissance itérée avec déflation sont :

1. Ces nombreux **produits matrice-vecteurs** qui augmentent le temps de calcul (notamment les calculs répétés de $A \cdot v$ dans la grande boucle).
2. Sa **dépendance aux types de matrice** utilisées.
3. **La limite de taille** de matrices pour le type 1 où la convergence n’a parfois pas lieu.
4. Et évidemment son **temps de calcul en général inférieur** à la fonction *eig* qui pourtant calcule tous les couples propres.

1.3 Extension de la puissance itérée au calcul des sous-espaces propres (*Subspace Iteration* v_0)

On veut maintenant étendre notre méthode de la puissance itérée au calcul d’un **bloc** de largeur m des couples propres dominants, et non plus un unique couple propre. Ce bloc est dès-lors

appelé un *sous-espace propre dominant*, et il est retrouvé à l'aide de m vecteurs V orthonormaux.

Cette méthode est appelée "l'itération de sous-espaces" ou "*Subspace Iteration*"; Voici son algorithme :

Algorithm 2 Subspace iteration method v0 (basic version)

Input : Symmetric matrix $A \in \mathbb{R}^{n \times n}$, number of required eigenpairs m , tolerance ε and *MaxIter* (max number of iterations)

Output : m dominant eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} .

Generate an initial set of m orthonormal vectors $V \in \mathbb{R}^{n \times m}$; $k = 0$

repeat

$k = k + 1$

$Y = A \cdot V$

$H = V^T \cdot A \cdot V$

Compute *accuracy* = $\|A \cdot V - V \cdot H\| / \|A\|$

$V \leftarrow$ orthonormalisation of the columns of Y

until ($k > \text{MaxIter}$ **or** *accuracy* $\leq \varepsilon$)

Compute the spectral decomposition $X \cdot \Lambda_{out} \cdot X^T = H$, where the eigenvalues of H (*diag*(Λ_{out})) are arranged in descending order of magnitude.

Compute the corresponding eigenspace $V_{out} = V \cdot X$

1.3.1 Converge de la matrice V

Appliqué à un ensemble de m vecteurs initiaux au lieu d'un seul, on attend de l'algorithme de la puissance itérée qu'il converge vers une **matrice V des vecteurs propres**. En particulier, chaque colonne/vecteur propre devrait correspondre à une valeur propre. Cependant, on remarque ici que ce n'est pas le cas puisque chaque colonne, à quelques signes près, représente en fait des vecteurs propres associés à une même valeur propre.

Cette erreur est corrigée dans *subspace_iter_v0* en **orthonormalisant** les colonnes de $Y = A \cdot V$ à la fin de chaque itération, d'où V sera bel et bien la matrice des vecteurs propres.

1.3.2 Impact du calcul de la décomposition spectrale de H

Alors que nous essayons d'éviter le calcul de la décomposition spectrale complète de A , l'Algorithme [2] effectue ce même calcul pour la matrice $H = V^T \cdot A \cdot V = X \cdot \Lambda_{out} \cdot X^T$. En effet, $V \in \mathbb{R}^{n \times m}$ et $A \in \mathbb{R}^{n \times n}$, d'où $H \in \mathbb{R}^{m \times m}$.

Or, les valeurs de m qu'on utilise sont **largement négligeables** devant n . C'est pour cela qu'on peut se permettre de calculer toute la décomposition spectrale de H sans causer de problème de temps de calcul.

1.3.3 Implémentation de l'algorithme 2

L'implémentation de l'algorithme d'itération des sous-espaces en *Matlab* est comme suit :

```
function [ V, D, it, flag ] = subspace_iter_v0( A, m, eps,
    maxit )
    % calcul de la norme de A (pour le critere de convergence)
    normA = norm(A, 'fro');
```



```

n = size(A,1);
% indicateur de la convergence
conv = 0;
% numero de l'iteration courante
k = 0;
% on genere un ensemble initial de m vecteurs orthogonaux
V = randn(n,m);
V = mgs(V);
% rappel : conv = invariance du sous-espace V : ||AV - VH
||/||A|| <= eps
while (~conv && k < maxit)
    k = k + 1;
    % calcul de Y = A.V
    Y = A * V;
    % calcul de H, le quotient de Rayleigh H = V^T.A.V
    H = V' * Y;
    % verification de la convergence
    conv = norm(Y - V*H, 'fro') <= eps*normA;
    % orthonormalisation
    V = mgs(Y);
end
% decomposition spectrale de H, le quotient de Rayleigh
[VH, WH] = eig(H);
% on range les valeurs propres dans l'ordre decroissant
[W, I] = sort(diag(WH), 'descend');
% on permute les vecteurs propres en consequence
X = VH(:,I);
% les m vecteurs propres dominants de A sont calcules a
partir de ceux de H
V = V * X;
D = diag(W);
it = k;
if (conv)
    flag = 0;
else
    flag = -3;
end
end
end

```

1.4 Amélioration en utilisant la projection de *Rayleigh-Ritz* (*Subspace Iteration* v_1)

On peut améliorer l'itération des sous-espaces afin d'en obtenir une implémentation efficace ; Cela va notamment se faire à l'aide de la Projection de Rayleigh-Ritz.

1.4.1 Concordance code-algorithme

L'algorithme suivant est annoté afin d'identifier les correspondances avec les fichiers de code rendus :

Algorithm 3 Subspace iteration method vI with Raleigh-Ritz projection

Input : Symmetric matrix $A \in \mathbb{R}^{n \times n}$, tolerance ε , $MaxIter$ (max nb of iterations) and $PercentTrace$ the target percentage of the trace of A
Output : n_{ev} dominant eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} .

Generate an initial set of m orthonormal vectors $V \in \mathbb{R}^{n \times m}$; {# lignes 48-49}
 $k = 0$; {# ligne 38}
 $PercentReached = 0$;
{# ligne 52}
repeat
 $k = k + 1$ {# ligne 54}
 Compute Y such that $Y = A \cdot V$; {# ligne 56}
 $V \leftarrow$ orthonormalisation of the columns of Y ; {# ligne 58}
 Rayleigh-Ritz projection applied on matrix A and orthonormal vectors V ; {# ligne 61}
 Convergence analysis step : save eigenpairs that have converged and update $PercentReached$ {# lignes 63-117 avec stockage en 91 et maj en 96}
until ($PercentReached > PercentTrace$ or $n_{ev} = m$ or $k > MaxIter$)

1.5 Accélération grâce à l'Approche par blocs (*Subspace Iteration* v_2)

Pour implémenter notre approche par blocs, nous avons besoin d'accélérer notre code ; Cela se fait simplement en remplaçant le produit $Y = A \cdot V$ par $Y = A^p \cdot V$, afin d'améliorer le coût en temps des orthonormalisations.

1.5.1 Coûts de calcul des matrices A^p et $A^p \cdot V$

Pour notre matrice $A \in \mathbb{R}^{n \times n}$, A^2 sera aussi de taille $n \times n$. Or, chaque élément du le produit matriciel $A \cdot A$ se calcule en n produits et $n - 1$ additions. Ainsi, on a $2n - 1$ flops pour un élément de la matrice. En multipliant avec la dimension carrée, on a donc $n^2(2n - 1)$ flops pour trouver A^2 .

Ainsi, on doit encore le faire $(p - 1)$ fois pour obtenir A^p , d'où on a : $(p - 1)n^2(n - 1)$ flops. Enfin, on multiplie cette matrice par le vecteur $V \in \mathbb{R}^{n \times m}$. On ajoute donc à notre nombre de flops, à partir des dimensions de V et du raisonnement précédent, $n \times m(2n - 1)$ flops pour le dernier produit. Cela fait donc $(p - 1)n^2(n - 1) + n \times m(2n - 1)$ flops.

Dans notre code, on peut organiser ce calcul différemment en initialisant A^p en dehors de la boucle puisque ni A ni p n'y sont modifiées. On économise donc énormément de puissance de calcul.

1.5.2 Implémentation de l'approche par bloc

Cette amélioration est intégrée de cette manière dans *subspace_iter_v2* :

```
A_p = A^p;  
% rappel : conv = (eigsum >= trace) | (nb_c == m)  
while (~conv && k < maxit)  
  
    k = k+1;  
    %% Y <- A^p*V  
    Y = A_p * Vr;  
    . . .
```

1.5.3 Comportement de la méthode selon la valeur de p

En augmentant graduellement p , on remarque en [1.3] que :

- le nombre d'itérations nécessaires commence d'abord par baisser jusqu'à un certain p ;
A partir de là, p est trop élevé et le nombre d'itérations nécessaires croît de nouveau. ;
- La qualité des vecteurs propres augmente quand p augmente ;

On peut observer ces opérations dans le tableau suivant pour des matrices de type 1 :

Matrices	p	Subspace Iteration v2
100×100	1	87
	3	29
	5	18
	7	6

TABLE 1.3 – Nombre d'itération pour converger selon la valeur p

On peut comparer les résultats de la v_2 avec la v_1 qui, en calculant $V = A \cdot V$ à chaque itération, est moins précis sur la qualité des vecteurs propres pour le même nombre d'opération.

1.6 Accélération et allègement avec la Méthode de Déflation (*Subspace Iteration* v_3)

Comme la convergence des colonnes de V se fait dans l'ordre, on peut "gêler" les vecteurs qui ont déjà convergé et ne plus y opérer. Cela nous fait gagner un temps important dans les différentes opérations principales de l'implémentation. Il s'agit de la méthode de déflation.

1.6.1 Différences dans la qualité des couples propres (q_v)

On peut évaluer q_v la qualité des couples propres pour la v_1 :

Ces différences de précision peuvent s'expliquer par le fait que, qu'importe leur état de convergence, les vecteurs sont continuellement recalculés au fil des itérations. Ainsi, les premiers

qv de la v_1
0.042235222240321
0.067268228558917
0.147699296959587
0.230402662274263
0.601875205878514
0.203258709325169

TABLE 1.4 – Valeurs de qv à l'échelle 10^{-7} .

vecteurs (accessoirement dominants et donc convergeant plus rapidement) seront en général plus qualitatifs en sortie du programme.

1.6.2 Prédications sur la précision des couples propres avec la méthode de déflation

Ainsi, à partir de la réponse précédente en [1.6.1], on peut prédire que *malgré le gain de temps considérable* dans la convergence des vecteurs propres, leur qualité/précision sera impactée négativement par leur "gel" vu qu'ils ne seront plus raffinés.

1.6.3 Implémentation de la méthode de déflation

On implémente la méthode de déflation comme suit :

```
while (~conv && k < maxit)

    k = k+1;

    %% Separation des vecteurs converges et non converges
    Vc = Vr(:, 1:nb_c); % Vecteurs converges
    Vnc = Vr(:, nb_c+1:m); % Vecteurs non converges

    if ~isempty(Vnc)
        %% Application de A^p uniquement sur les vecteurs non
        converges
        Y_nc = A^p * Vnc;

        %% Orthogonalisation avec mgs_block
        if ~isempty(Vc)
            % Concatener Vc et Y_nc pour utiliser mgs_block
            Y = [Vc, Y_nc];

            % Utiliser mgs_block avec nb_c comme parametre pour
            indiquer
```

```

        % que les nb_c premieres colonnes sont deja
        orthonormales
        Y = mgs_block(Y, nb_c);

        % Extraire Y_nc mis a jour
        Y_nc = Y(:, nb_c+1:end);
    else
        % Si pas de vecteurs converges, orthogonaliser Y_nc
        directement
        Y_nc = mgs(Y_nc);
    end

    %% Reconstitution de Y = [Vc, Y_nc]
    Y = [Vc, Y_nc];

    % Mise a jour de Vr
    Vr = Y;

    %% Projection de Rayleigh-Ritz uniquement sur les
    colonnes non convergees
    if nb_c < m
        % Calcul de la projection de Rayleigh-Ritz pour les
        vecteurs non converges
        [Wr_nc, Vr_nc] = rayleigh_ritz_projection(A, Y_nc);

        % Mise a jour des vecteurs non converges dans Vr
        Vr(:, nb_c+1:m) = Vr_nc;

        % Conservation des valeurs de Wr pour les colonnes
        non convergees
        Wr = [W(1:nb_c); Wr_nc];
    else
        Wr = W;
    end
else
    % Tous les vecteurs ont converge
    Wr = W;
end
end

```

1.7 Expérimentations

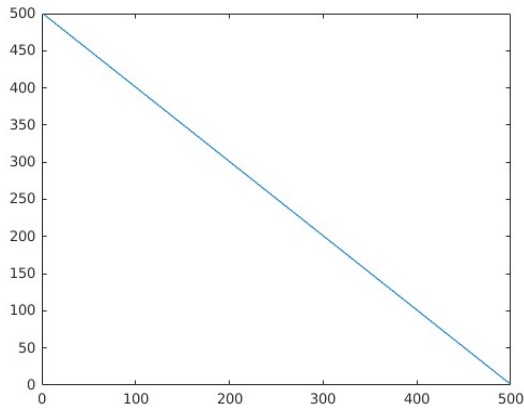
1.7.1 Types de matrices utilisées et comparaison selon la distribution des couples propres

A partir du fichier *matgen_csad*, on sait que les types de matrice sont :

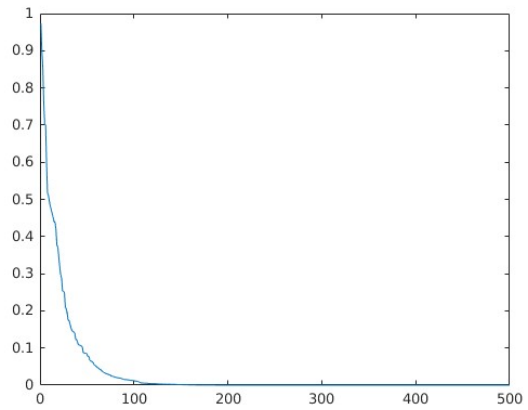
- **type 1** : Matrice dont les valeurs propres sont égales à leur indice ($D_i = i$).
- **type 2** : Matrice dont la distribution des valeurs propres est aléatoire suivant une loi

uniforme sur les logarithmes.

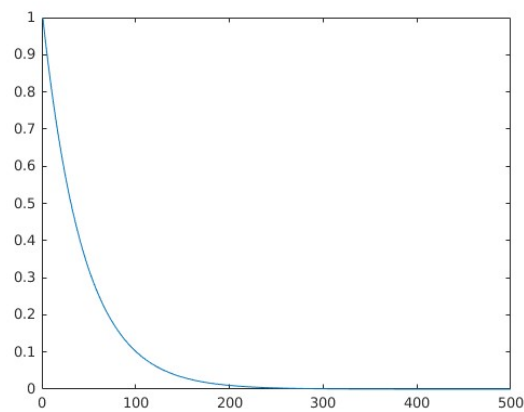
- **type 3** : Matrice dont la distribution des valeurs propres suit la formule $cond^{-\frac{(i-1)}{n-1}}$ avec $cond = 10^5$.
- **type 4** : Matrice dont la distribution des valeurs propres suit la formule $1 - \frac{i-1}{n-1} \times (1 - \frac{1}{cond})$ avec $cond = 10^2$.



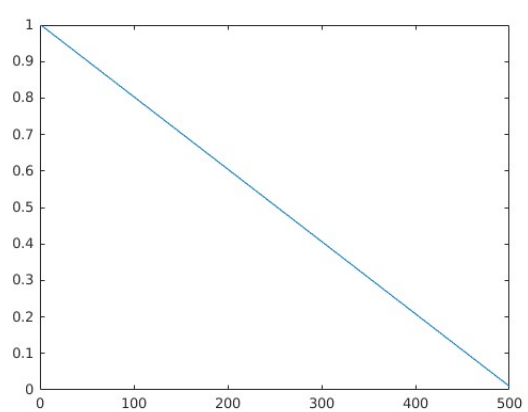
(a) Type 1



(b) Type 2



(c) Type 3



(d) Type 4

FIGURE 1.1 – Distribution des valeurs propres pour une matrice A de taille 500×500

1.7.2 Comparaison complète des méthodes implémentées avec celles fournies

On peut comparer les performances temporelles de nos différentes méthodes :

Matrices		v0	v1	v2	v3	eig
100 100	Type 1	4.6×10^{-1}	8×10^{-2}	2×10^{-2}	3×10^{-2}	1×10^{-2}
	Type 2	2×10^{-2}	1×10^{-2}	1×10^{-2}	3×10^{-1}	2×10^{-2}
	Type 3	5×10^{-2}	1×10^{-2}	8×10^{-2}	1×10^{-2}	1×10^{-2}
	Type 4	5×10^{-1}	9×10^{-2}	1×10^{-2}	3×10^{-2}	1×10^{-2}
500 500	Type 1	6.44	non convergé	non convergé	non convergé	6×10^{-2}
	Type 2	2.3×10^{-1}	5×10^{-2}	4×10^{-2}	1×10^{-1}	5×10^{-2}
1000 1000	Type 1	2.2	non convergé	non convergé	non convergé	3.1×10^{-1}

TABLE 1.5 – Comparaison des performances temporelles des différents algorithmes de recherche de couples propres.

On en conclut donc que :

- L'algorithme le plus fiable est la v0 car sûr de converger malgré ces temps de convergence bien plus longs.
- Les algorithmes v1, v2, v3 voient leur vitesse de calculs largement améliorée par rapport à la version qui les précède. Cependant, ils ne sont pas certains de converger, à l'inverse de la v0.
- Vis-à-vis de la précision des valeurs propres trouvées, la v3 a beau être la plus rapide, elle perd en qualité. On peut donc se demander selon le contexte quel algorithme est le plus approprié à utiliser, mais on peut aussi confirmer que nos améliorations de la puissance itérée vers les différentes itérations de sous-espace a été un succès.

Chapitre 2

Application à la compression d'images

Notre but est donc d'appliquer ces nouveaux algorithmes à la compression d'image.

2.1 Question 1

Σ_k est une matrice de dimension $k \times k$, U_k est de dimension $q \times k$ et V_k est de dimension $p \times k$, avec p et q les deux dimensions de l'image et k le nombre de vecteur utilisés pour sa reconstruction.

2.2 Question 2

Les tests présentés sont réalisé sur la même image mais les programmes utilisés sont fonctionnels pour toutes les images mises à dispositions.

2.2.1 Fonction eig

Nous avons tout d'abord mis en place la deuxième alternative dans laquelle nous utilisons la fonction eig de Matlab.

Avec cette fonction, on obtient 944 valeurs propres en, en moyenne, 0,07s

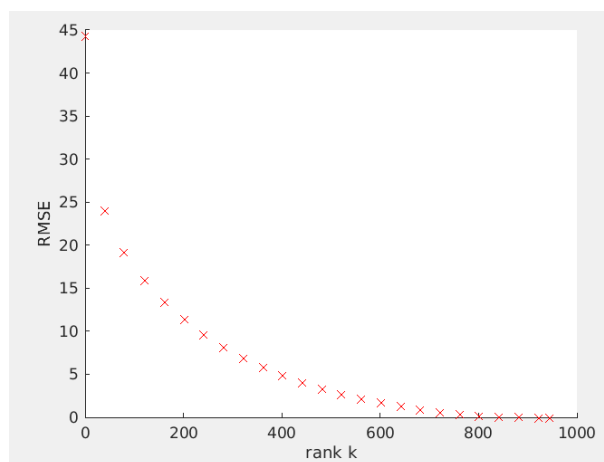


FIGURE 2.1 – RMSE de l'image reconstruite

2.2.2 Méthode de la puissance itérée

Nous avons ensuite utilisé la méthode de la puissance itérée pour calculer les couples propres. En faisant varier les paramètres de cette méthode, pour une même image, nous obtenons les résultats suivant :

<i>eps</i>	<i>maxit</i>	pourcentage	nombre de valeurs propres calculées	durée de calcul
10^{-8}	10000	99%	50	3,5s
10^{-8}	10000	99,5%	119	17,5s
10^{-8}	100000	99,9%	312	80s
10^{-6}	10000	99,5%	119	12,3s
10^{-3}	10000	99,5%	119	3,1s

On remarque que plus on augmente le pourcentage recherché de la trace matrice, plus le temps de calcul est élevé, atteignant de très longue durée pour un pourcentage proche de 100%. Pour faire baissé ce temps de calcul, on peut augmenté la valeur de eps. En effet, plus sa valeur est grande, plus le temps de calcul est faible. Cependant, cela augmente l'erreur lors de la reconstitution de l'image (ce qui semble logique car les couples propres sont calculés avec moins de précision).

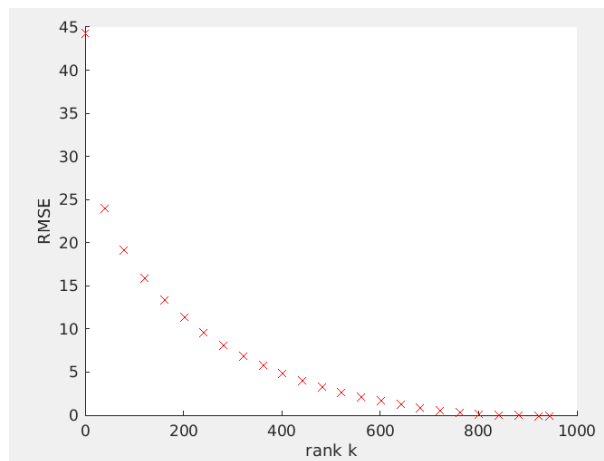


FIGURE 2.2 – RMSE de l'image reconstruite

2.2.3 Subspace Iteration Method, Version 0

Nous avons ensuite utilisé les différentes version de la "subspace iteration method". Voici les résultats obtenus :

eps	$maxit$	$search_space$	durée de calcul
10^{-4}	100	200	0,2s
10^{-6}	100	200	1,2s
10^{-8}	100	200	3,7s
10^{-4}	100	900	2,4s
10^{-6}	300	900	2,5s
10^{-6}	1000	900	2,5s

Comme précédemment, un eps grand permet de réduire le temps de calcul au détriment de la précision. Le nombre maximum d'itérations $maxit$ a l'effet inverse. La variable $search_space$ permet de définir le nombre de couple propres à calculer.

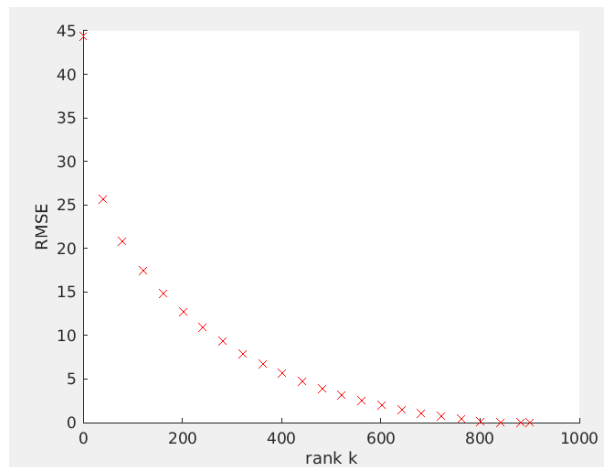


FIGURE 2.3 – RMSE de l'image reconstruite, $eps = 10^{-6}$, $maxit = 300$, $search_space = 900$

2.2.4 Subspace Iteration Method, Version 1

$maxit$	$search_space$	$pourcentage$	durée de calcul
100	300	99,9%	0,9s
100	900	99.99%	1,7s
100	900	100%	1,8s
avec $eps = 10^{-6}$			

On remarque que pour des pourcentages très élevés, les temps de calcul ne sont pas aussi élevés qu'avec les fonctions précédentes.

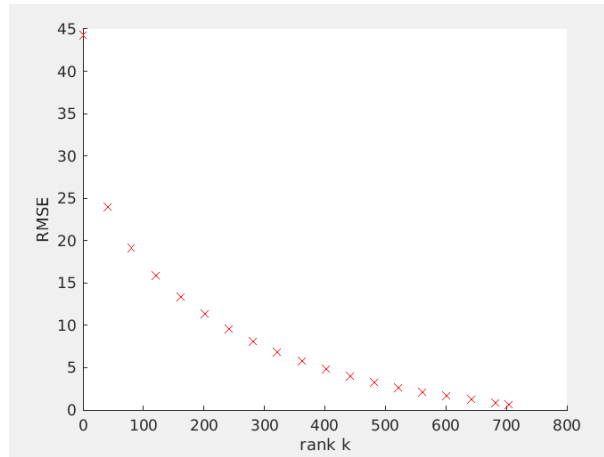


FIGURE 2.4 – RMSE de l’image reconstruite avec $search_space = 900$ et $pourcentage = 100\%$

2.2.5 Subspace Iteration Method, Version 2

<i>maxit</i>	<i>search_space</i>	<i>pourcentage</i>	<i>puiss</i>	durée de calcul
100	300	99,9%	1	0,9s
100	900	100%	1	2s
100	900	99,9%	2	2s
100	900	99,9%	3	2,7s

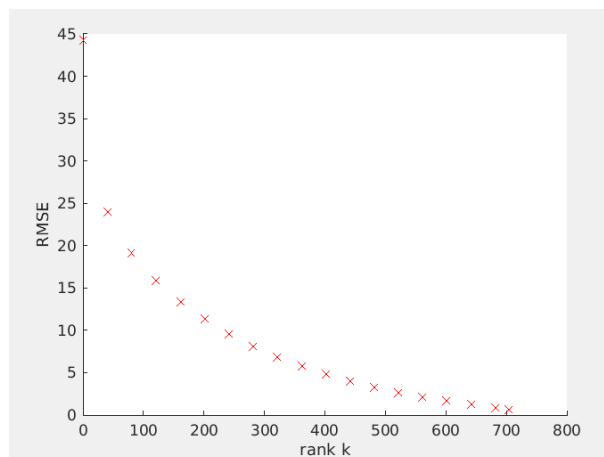


FIGURE 2.5 – RMSE de l’image reconstruite avec $search_space = 900$ et $pourcentage = 99,99\%$, $puiss = 1$

L’argument $puiss$ augmente le temps de calcul mais l’impact sur le RMSE est minime.

2.2.6 Subspace Iteration Method, Version 3

<i>maxit</i>	<i>search_space</i>	<i>pourcentage</i>	<i>puiss</i>	durée de calcul
100	900	99,9%	1	1,9s
100	900	100%	1	2s
100	900	99,9%	2	2s
100	900	99,9%	3	2,8s

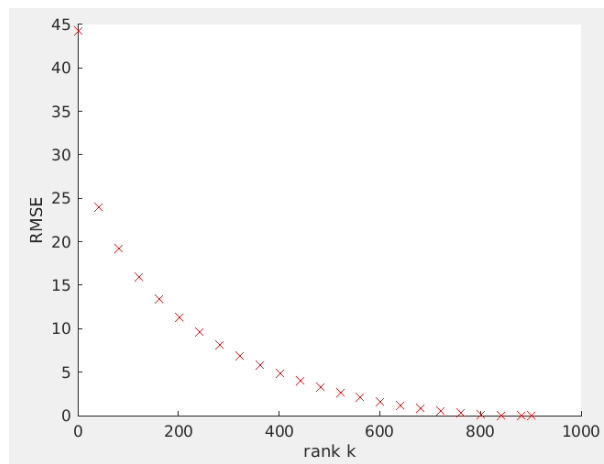


FIGURE 2.6 – RMSE de l'image reconstruite avec $search_space = 900$ et $pourcentage = 99,99\%$, $puiss = 1$

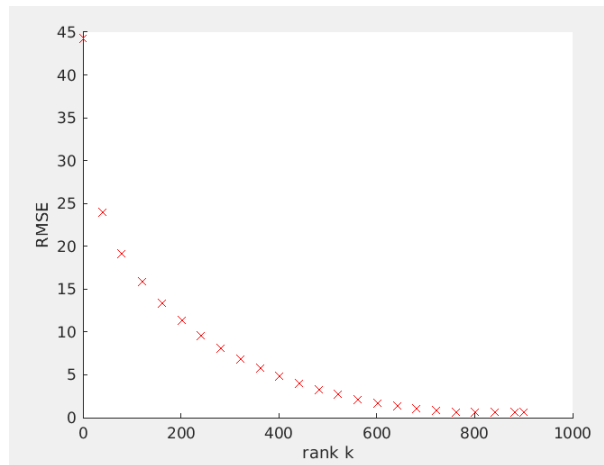


FIGURE 2.7 – RMSE de l'image reconstruite avec $search_space = 900$ et $pourcentage = 99,99\%$, $puiss = 3$

On remarque que la RMSE est plus légèrement plus faible pour $puiss = 1$.

Conclusion

A partir de l'information qu'il n'était pas forcément nécessaire de calculer l'entièreté de la décomposition spectrale d'une matrice pour en exploiter les informations, nous avons avec succès implémenté différentes méthodes de convergence vers les couples propres voulus. Chaque amélioration fait mieux temporellement que la précédente mais vient avec un lot de limitations que nous avons détaillé.

Ces méthodes, plus ou moins rapides et plus ou moins qualitatives/précises, ont pu être appliquées de manière encourageante à la compression d'image.